Journal of Combinatorial Optimization (2021) 41:579–602
https://doi.org/10.1007/s10878-020-00673-2

Length-weighted λ-rearrangement distance

Alexsandro Oliveira Alexandrino[1] · **Guilherme Henrique Santos Miranda**[1] · **Carla Negri Lintzmayer**[2] · **Zanoni Dias**[1]

Accepted: 6 November 2020 / Published online: 18 November 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020, corrected publication 2021

Abstract
Comparative genomics is a field of biology that aims at comparing genomes of different species. One major question of this field is to find the evolutionary distance between two given genomes. One way to estimate such distance is to use the rearrangement distance, which consists in finding a minimum cost sequence of rearrangements that transforms one genome into another. We use permutations to model the genomes being compared and, in this way, we can treat this problem as the problem of sorting a permutation with a minimum cost sequence of rearrangements. In the early works with rearrangement distance, it was considered that all rearrangements are equally likely to occur and, consequently, they use a unitary cost for all rearrangements. Some variations of the problem were motivated by the observation that rearrangements involving large segments of a genome rarely occur. One of these variants also uses a unitary cost, however it adds a constraint in the length of the operations allowed to estimate the distance. Another variant uses a cost function based on the rearrangement's length. In this work, we study problems that combine both variants, that is, problems with a constraint in the rearrangement's length and with a cost function based on the rearrangement's length. We present approximation algorithms for five such problems involving reversals and/or transpositions for sorting signed and unsigned permutations. We also analyze the problems for specific parameters of the length restriction and for when the cost function is equal to ℓ^α, where ℓ is the rearrangement's length and $\alpha \geq 1$ is a real value parameter.

Keywords Genome rearrangements · Approximation algorithms · Sorting permutations

A preliminary version of this paper previously appeared in the Proceedings of the "X Latin and American Algorithms, Graphs and Optimization Symposium (LAGOS'2019)" (Alexandrino et al. 2019).
Data sharing not applicable to this article as no datasets were generated or analysed during the current study.

Extended author information available on the last page of the article

1 Introduction

One of the main challenges in Computational Biology is to estimate the *evolutionary distance* among species, considering that new organisms arise from mutations that occurred on the genetic material of other organisms. One way to estimate such distance is to consider the *rearrangement distance* between the organisms' genomes we are comparing. Such distance consists in the cost of the sequence of rearrangements that occurred in the transformation of one genome into the other.

A *genome rearrangement* is a large scale mutation which alters stretches of a genome. Two of the most studied genome rearrangements are reversals and transpositions. A reversal reverses the order of the genes in a segment of the genome, while a transposition exchanges two adjacent segments. We define the *length* of a rearrangement as the number of elements affected by it.

A genome can be represented as an ordered set of genes. A *synteny block* is a region with high similarity between two distinct genomes, and it can be a single gene or a sequence of genes. When we want to compare two genomes, we represent a genome as a sequence of synteny blocks (Dias 2012). Assuming that there are no repeated genes, this representation is modeled as a permutation of integers. Furthermore, genes and blocks have orientations and, when this orientation is known, the genome is modeled as a *signed permutation*, where the sign of an element indicates the orientation of the gene or block. When the orientation is unknown, we simply use an *unsigned permutation* to represent a genome.

When using permutations, the genome rearrangement problems consist in finding a minimum cost sequence of rearrangements which transforms one permutation into another. We can always represent one of the genomes as the identity permutation (sorted permutation) and, in this way, these problems are equivalent to sorting a permutation with a minimum cost sequence of rearrangements.

The problems of Sorting Permutations by Reversals and Sorting Permutations by Transpositions are NP-hard (Caprara 1999; Bulteau et al. 2012). The best result for both problems is a 1.375-approximation algorithm (Berman et al. 2002; Elias and Hartman 2006). When considering only signed permutations, the problem of Sorting Permutations by Reversals becomes solvable in polynomial time (Hannenhalli and Pevzner 1999). The problem of Sorting Permutations by (Signed or Unsigned) Reversals and Transpositions is also NP-hard (Oliveira et al. 2019). For this problem, the best results are a 2-approximation algorithm (Walter et al. 1998) for signed permutations and a $2k$-approximation algorithm (Rahman et al. 2008) for unsigned permutations, where k is the approximation factor for cycle decomposition of breakpoints graphs. Using the best-known value of k (Chen 2013), this approximation factor becomes equal to $2.8334 + \epsilon$, for $\epsilon > 0$.

All the problems mentioned above consider the traditional approach of sorting permutations by rearrangements problems. In this approach, all rearrangements are considered to have the same probability to happen and, consequently, the rearrangement distance is equal to the minimum number of rearrangements that sort a permutation. However, studies show that some rearrangements are more likely to occur than others (Bader and Ohlebusch 2007; Blanchette et al. 1996) and, therefore, a weighted approach is more appropriate to estimate the evolutionary distance. In the

literature, there are cost functions based on the rearrangement's length (Pinter and Skiena 2002; Nguyen et al. 2005; Lintzmayer et al. 2015) and on the rearrangement's type (Alexandrino et al. 2018, 2020; Bader and Ohlebusch 2007; Blanchette et al. 1996; Eriksen 2002).

The problem of Sorting Permutations by Length-Weighted Rearrangements was motivated by the observation that, in some species, the rearrangements that occurred in the evolutionary process do not act on very large regions (Blanchette et al. 1996; Lefebvre et al. 2003). In this variation, the cost of a rearrangement β is defined by the function $f(\beta) = |\beta|^\alpha$, where $|\beta|$ is the length of β and $\alpha > 0$ is a real value parameter. For $\alpha = 1$, the Sorting by Length-Weighted (Signed or Unsigned) Reversals has a $O(\lg n)$-approximation algorithm (Bender et al. 2008). Considering transpositions and the combination of (signed or unsigned) reversals and transpositions, the best known results are $O(\lg^2 n)$-approximation algorithms (Lintzmayer et al. 2015). For $\alpha \geq 2$, there are algorithms with constant approximation factor for all variations (Bender et al. 2008; Lintzmayer et al. 2015). Furthermore, for $\alpha \geq 3$, the problems involving unsigned permutations become solvable in polynomial time (Bender et al. 2008).

Another variant of the genome rearrangement problems, where there exists a constraint in the length of the allowed rearrangements, arose from the same motivation of the length-weighted variation. Given an integer $\lambda > 1$, only rearrangements of length less than or equal to λ are allowed to modify a permutation in the problems of Sorting Permutations by λ-Rearrangements. For any value of λ and any combination of rearrangements (reversals and/or transpositions) for signed and unsigned permutations, there are $O(\lambda^2)$-approximation algorithms (Miranda et al. 2019). For $\lambda = 3$, there exist algorithms with constant approximation factor (Galvão et al. 2015; Vergara 1998). For $\lambda = 2$, these problems are solvable in polynomial time (Jerrum 1985; Galvão et al. 2015).

Nguyen et al. (2005) considered the problem of Sorting Length-Weighted Signed λ-Reversals, which combines the length constraint with the cost function based on the length of the rearrangements. They presented a $O(\lg n)$-approximation, for $\lambda = \Omega(n)$ and $\alpha = 1$, and a $(2\lg^2 n + \lg n)$-approximation, for $\lambda = o(n)$ and $\alpha = 1$.

In this work, we study the problems of Sorting Permutations by Length-Weighted λ-Rearrangements, which are named as: Sorting Permutations by Length-Weighted λ-Reversals (SbR); Sorting Permutations by Length-Weighted λ-Transpositions (SbT); Sorting Permutations by Length-Weighted λ-Reversals and λ-Transpositions (SbRT); Sorting Permutations by Length-Weighted Signed λ-Reversals (Sb$\bar{\text{R}}$); and Sorting Permutations by Length-Weighted Signed λ-Reversals and λ-Transpositions (Sb$\bar{\text{R}}$T).

We start by considering $\alpha = 1$ and any value of λ. After this, yet considering $\alpha = 1$, we present results (i) for large values of λ and (ii) for $\lambda = 3$. A rearrangement of length at most 3 is also known as a short rearrangement (Heath and Vergara 2003; Galvão et al. 2015). At last, we consider that $\alpha > 1$ and analyze how this affects the rearrangement distance.

The rest of this work is organized as follows. Section 2 presents definitions related to the problems. Sections 3 and 4 describe algorithms for unsigned permutations and Sect. 5 describes algorithms for signed permutations. Section 6 presents new approximation factors for large values of λ. Section 7 considers the problem of Sorting Permutations by Length-Weighted Short Rearrangements. Section 8 analyses the

problems when using $\alpha > 1$. At last, Sect. 9 presents final considerations and future work.

2 Definitions

A *signed permutation* of length n is represented as $\pi = (\pi_1\ \pi_2\ \ldots\ \pi_n)$, such that $\pi_i \in \{-n, \ldots, -2, -1, +1, +2, \ldots, +n\}$ and $|\pi_i| \neq |\pi_j|$ if and only if $i \neq j$, for all i and j. An *unsigned permutation* of length n is represented similarly, but $\pi_i \in \{1, 2, \ldots, n\}$ and $\pi_i \neq \pi_j$ if and only if $i \neq j$, for all i and j. The *identity permutation*, $\iota = (1\ 2\ \ldots\ n)$, is the goal of the sorting problems we will study. For signed permutations, all elements of ι have positive sign.

A *signed reversal* $\bar{\rho}(i, j)$, with $1 \leq i \leq j \leq n$, reverts the segment $\pi_i, \pi_{i+1}, \ldots, \pi_j$ and flips the sign of each affected element. An *unsigned reversal* $\rho(i, j)$, with $1 \leq i < j \leq n$, only reverts the segment $\pi_i, \pi_{i+1}, \ldots, \pi_j$. So, when π is a signed permutation, we have that $(\pi_1\ \ldots\ \pi_{i-1}\ \underline{\pi_i\ \ldots\ \pi_j}\ \pi_{j+1}\ \ldots\ \pi_n) \cdot \bar{\rho}(i, j) = (\pi_1\ \ldots\ \pi_{i-1}\ \underline{-\pi_j\ \ldots\ -\pi_i}\ \pi_{j+1}\ \ldots\ \pi_n)$ and, when π is an unsigned permutation, we have that $(\pi_1\ \ldots\ \pi_{i-1}\ \underline{\pi_i\ \ldots\ \pi_j}\ \pi_{j+1}\ \ldots\ \pi_n) \cdot \rho(i, j) = (\pi_1\ \ldots\ \pi_{i-1}\ \underline{\pi_j\ \ldots\ \pi_i}\ \pi_{j+1}\ \ldots\ \pi_n)$. The length of a signed reversal $\bar{\rho}(i, j)$ or an unsigned reversal $\rho(i, j)$ is equal to $|\bar{\rho}(i, j)| = |\rho(i, j)| = j - i + 1$.

A *transposition* $\tau(i, j, k)$, with $1 \leq i < j < k \leq n + 1$, exchanges the two adjacent segments $\pi_i, \pi_{i+1}, \ldots, \pi_{j-1}$ and $\pi_j, \pi_{j+1}, \ldots, \pi_{k-1}$ without changing signs of their elements. Therefore, we have that $(\pi_1\ \ldots\ \pi_{i-1}\ \underline{\pi_i\ \ldots\ \pi_{j-1}}\ \underline{\pi_j\ \ldots\ \pi_{k-1}}\ \pi_k\ \ldots\ \pi_n) \cdot \tau(i, j, k) = (\pi_1\ \ldots\ \pi_{i-1}\ \underline{\pi_j\ \ldots\ \pi_{k-1}}\ \underline{\pi_i\ \ldots\ \pi_{j-1}}\ \underline{\pi_k\ \ldots\ \pi_n})$. A transposition $\tau(i, j, k)$ has length $|\tau(i, j, k)| = k - i$.

Given an integer $\lambda > 1$, we say that a rearrangement is a λ-rearrangement if its length is less than or equal to λ. Furthermore, we say that a rearrangement β is *super short* if $|\beta| \leq 2$ and *short* if $|\beta| \leq 3$.

Given a real value $\alpha \geq 0$, in the length-weighted approach, a rearrangement β has cost $f(\beta) = |\beta|^{\alpha}$. For a sequence of rearrangements $S = \langle \beta_1, \beta_2, \ldots, \beta_m \rangle$, we have that $f(S) = \sum_{i=1}^{m} f(\beta_i)$. Note that the unweighted approach is equivalent to setting $\alpha = 0$. In this work, we consider that $\alpha = 1$, unless we explicitly say otherwise (Sect. 8), when we analyze other values of α.

A *rearrangement model* is the set of allowed rearrangements for a specific sorting problem. Given an integer λ, a rearrangement model M with only λ-rearrangements, and a permutation π, the λ-*rearrangement distance* $c_M^{\lambda}(\pi)$ is equal to $\sum_{i=1}^{m} f(\beta_i)$ such that every rearrangement $\beta_1, \beta_2, \ldots, \beta_m$ is in M, $\pi \cdot \beta_1 \cdot \beta_2 \ldots \cdot \beta_m = \iota$, and $\sum_{i=1}^{m} f(\beta_i)$ is minimum.

We use $r, \bar{r}, t, rt, \bar{r}t$ to denote unsigned λ-reversals, signed λ-reversals, λ-transpositions, unsigned λ-reversals and λ-transpositions, and signed λ-reversals and λ-transpositions, respectively. For instance, the signed λ-reversals distance of the permutation π is denoted by $c_{\bar{r}}^{\lambda}(\pi)$.

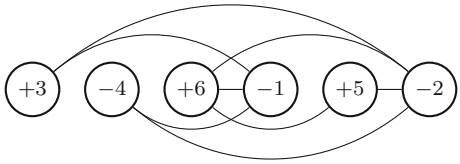

Fig. 1 Permutation graph $G(\pi)$ of the signed permutation $\pi = (+3\ -4\ +6\ -1\ +5\ -2)$. In this example, $c(\pi) = c_{\text{odd}}(\pi) = 1$ and $\gamma(\pi) = 6$

2.1 Inversions

A pair of elements (π_i, π_j) of π is called an *inversion* if $i < j$ and $|\pi_i| > |\pi_j|$. We use $\text{Inv}(\pi)$ to denote the number of inversions in a permutation π. For any permutation π, $\text{Inv}(\pi) = 0$ if and only if $|\pi_1| < |\pi_2| < \cdots < |\pi_n|$. Note that considering unsigned permutations, only the identity permutation has $\text{Inv}(\pi) = 0$.

Example 1 The permutation $\pi = (3\ 4\ 1\ 2)$ has the inversions $(\pi_1 = 3, \pi_3 = 1)$, $(\pi_1 = 3, \pi_4 = 2)$, $(\pi_2 = 4, \pi_3 = 1)$, and $(\pi_2 = 4, \pi_4 = 2)$. In this example, $\text{Inv}(\pi) = 4$.

Lemma 1 *For any permutation π, if $\text{Inv}(\pi) > 0$, then there exists an inversion (π_i, π_{i+1}).*

Proof Let $\pi_1, \ldots, \pi_i$ be a maximal subsequence such that $|\pi_j| < |\pi_{j+1}|$ for all $1 \leq j < i$. Since $\text{Inv}(\pi) > 0$, we have that $i < n$. So, $|\pi_i| > |\pi_{i+1}|$ and (π_i, π_{i+1}) is an inversion. □

Given a permutation π and a rearrangement β, let $\Delta\text{Inv}(\pi, \beta) = \text{Inv}(\pi) - \text{Inv}(\pi \cdot \beta)$, that is, $\Delta\text{Inv}(\pi, \beta)$ denotes the change caused by β in the number of inversions in π. From Lemma 1, we can conclude that for any permutation π such that $\text{Inv}(\pi) > 0$, there exists a super short rearrangement β with $\Delta\text{Inv}(\pi, \beta) = 1$.

The *permutation graph* or *inversion graph* of a permutation π is the undirected graph $G(\pi) = (V, E)$, where $V = \{\pi_1, \pi_2, \ldots, \pi_n\}$ and $E = \{(\pi_i, \pi_j)$: the pair (π_i, π_j) is an inversion in $\pi\}$. Figure 1 shows an example of a permutation graph.

Given a signed permutation π, a connected component C of $G(\pi)$ is *odd* if it contains an odd number of negative elements (vertices) and it is *even* otherwise. We use $c(\pi)$ and $c_{\text{odd}}(\pi)$ to denote the number of connected components and the number of odd connected components of $G(\pi)$, respectively. An edge e of $G(\pi)$ whose deletion increases the number of connected components of $G(\pi)$ is called a *cut-edge*. We use $\gamma(\pi)$ to denote the number of non-isolated vertices in $G(\pi)$, that is, the number of elements in π belonging to at least one inversion.

2.2 Entropy

Given a permutation π, the *entropy* of π_i is defined as $\text{ent}(\pi_i) = ||\pi_i| - i|$, for $1 \leq i \leq n$. In other words, the entropy of an element π_i is the distance between π_i and its position in the identity permutation. We define the entropy of a permutation π as the sum of the entropy of its elements. Formally, the entropy of the permutation π is denoted as $\text{ent}(\pi) = \sum_{i=1}^{n} \text{ent}(\pi_i)$.

Algorithm 1: Inversion based greedy algorithms.

Input: An unsigned permutation π, a value $\lambda > 1$, and a model $\mathcal{M}$ containing only λ-rearrangements.
Output: A sequence of rearrangements from $\mathcal{M}$ that sorts π.

```
Let S be an empty sequence
while Inv(π) > 0 do
    Let β be the operation from M such that the ratio ΔInv(π, β)/f(β) is maximum
    Apply β in π
    Append β to S
return S
```

For any permutation π, $\text{ent}(\pi) = 0$ if and only if $|\pi_1| < |\pi_2| < \cdots < |\pi_n|$. Note that considering unsigned permutations, only the identity permutation has $\text{ent}(\pi) = 0$.

Given a permutation π and a rearrangement β, let $\Delta\text{ent}(\pi, \beta) = \text{ent}(\pi) - \text{ent}(\pi \cdot \beta)$, that is, $\Delta\text{ent}(\pi, \beta)$ denotes the change caused by β in the entropy of the permutation π. We also use $\Delta\text{ent}(\pi_i, \beta)$ to denote the change caused by β in the entropy of the element π_i, with $1 \leq i \leq n$.

We define the sets $E_\pi^{\text{even}^-} = \{\pi_i : \pi_i < 0 \text{ and } \text{ent}(\pi_i) \text{ is even}\}$ and $E_\pi^{\text{odd}^+} = \{\pi_i : \pi_i > 0 \text{ and } \text{ent}(\pi_i) \text{ is odd}\}$. These sets will be used in lower bounds for $\overline{\text{SbR}}$ and $\overline{\text{SbRT}}$.

Example 2 The permutation $\pi = (+5\ -4\ +3\ -1\ +2)$ has $\text{ent}(+5) = 4$, $\text{ent}(-4) = 2$, $\text{ent}(+3) = 0$, $\text{ent}(-1) = 3$, $\text{ent}(+2) = 3$, and $\text{ent}(\pi) = 12$. Besides that, we have the sets $E_\pi^{\text{even}^-} = \{-4\}$ and $E_\pi^{\text{odd}^+} = \{+2\}$.

Next, we introduce a result presented by Galvão et al. (2015) that will be used in the algorithms for signed permutations.

Lemma 2 (Galvão et al. 2015) *For any signed permutation π and signed reversal $\bar{\rho}$ such that $|\bar{\rho}| = 2$, we have that $|E_\pi^{\text{even}^-}| + |E_\pi^{\text{odd}^+}| = |E_{\pi'}^{\text{even}^-}| + |E_{\pi'}^{\text{odd}^+}|$, where $\pi' = \pi \cdot \bar{\rho}$.*

3 Inversions based algorithms for unsigned permutations

In this section, we focus on SbR, SbT, and SbRT problems. Algorithm 1 presents the general idea of the inversion based greedy algorithms, whose choice is to prioritize rearrangements that most reduce the number of inversions with the lowest cost.

Consider the unsigned permutation $\pi = (4\,5\,2\,3\,1)$, $\lambda = 3$, and the model containing λ-reversals. The algorithm applies the following sequence of λ-reversals over π, which has a cost of 10:

$$\begin{aligned}
\pi^0 &= (4\ 5\ 2\ 3\ 1) = \pi \\
\pi^1 &= \pi^0 \cdot \rho(2, 3) = (4\ 2\ 5\ 3\ 1) \\
\pi^2 &= \pi^1 \cdot \rho(3, 5) = (4\ 2\ 1\ 3\ 5) \\
\pi^3 &= \pi^2 \cdot \rho(1, 3) = (1\ 2\ 4\ 3\ 5)
\end{aligned}$$

$$\pi^4 = \pi^3 \cdot \rho(3,4) = \iota$$

The permutation π has 8 inversions. The λ-reversals $\rho(2,3)$, $\rho(4,5)$, $\rho(1,3)$, $\rho(2,4)$, and $\rho(3,5)$ decrease the number of inversions by one. The two first λ-reversals have a cost of 2 and the remaining ones have a cost of 3 each. So, in the first iteration, $\rho(2,3)$ and $\rho(4,5)$ are the λ-reversals with maximum ratio. At this point, the algorithm can choose either one of them and, in this example, the algorithm arbitrarily chose the λ-reversal $\rho(2,3)$. The algorithm continues, at each iteration, applying the λ-reversal with maximum ratio and, eventually, the permutation is equal to the identity.

To prove the approximation factors, Lemmas 3 and 4 present lower bounds for the SbT, SbR, and SbRT problems relating the sorting distance to the number of inversions in a permutation, and Lemma 5 presents upper bounds for the cost of the solutions constructed by the inversion based algorithms.

Lemma 3 *Given a permutation π and an integer λ, any λ-reversal ρ has ratio $\Delta\mathrm{Inv}(\pi,\rho)/|\rho| \leq (\lambda-1)/2$ and any λ-transposition τ has ratio $\Delta\mathrm{Inv}(\pi,\tau)/|\tau| \leq \lambda/4$.*

Proof A reversal $\rho(i,j)$ of length k can affect only the inversions in the interval from i to j. Since there are at most $k(k-1)/2$ inversions in this interval, we have that

$$\frac{\Delta\mathrm{Inv}(\pi,\rho)}{|\rho|} \leq \frac{k(k-1)/2}{k} = \frac{k-1}{2}.$$

Therefore, we conclude that $\Delta\mathrm{Inv}(\pi,\rho)/|\rho| \leq (\lambda-1)/2$, since we are considering λ-reversals.

A transposition $\tau(i,j,k)$ of length k' can affect only the inversions in the interval from i to $k-1$. Moreover, this transposition does not affect inversions between the elements in the intervals from i to $j-1$ and from j to $k-1$. In this way, an inversion (π_x, π_y) is removed only if $x \in [i,j)$ and $y \in [j,k)$. Therefore,

$$\frac{\Delta\mathrm{Inv}(\pi,\tau)}{|\tau|} \leq \frac{(j-i)(k-j)}{k'} \leq \frac{(k'/2)^2}{k'} = \frac{k'}{4},$$

since $(j-i)+(k-j) = k'$. Considering only λ-transpositions, we conclude that $\Delta\mathrm{Inv}(\pi,\tau)/|\tau| \leq \lambda/4$. □

Lemma 4 *For any permutation π and value $\lambda > 1$, $c_r^\lambda(\pi) \geq c_{rt}^\lambda(\pi) \geq 2\mathrm{Inv}(\pi)/(\lambda-1)$ and $c_t^\lambda(\pi) \geq 4\mathrm{Inv}(\pi)/\lambda$.*

Proof Since any sorting sequence for the SbR problem is a valid sorting sequence for SbRT, we have that $c_r^\lambda(\pi) \geq c_{rt}^\lambda(\pi)$. By Lemma 3, for any λ-rearrangement β, the ratio $\Delta\mathrm{Inv}(\pi,\beta)/|\beta|$ is at most $(\lambda-1)/2$, which means that the minimum cost to decrease $\mathrm{Inv}(\pi)$ by 1 is $2/(\lambda-1)$. Since the identity permutation has $\mathrm{Inv}(\iota) = 0$, any sorting sequence needs to decrease $\mathrm{Inv}(\pi)$ inversions in π to sort the permutation, resulting in the lower bound $2\mathrm{Inv}(\pi)/(\lambda-1)$.

Similarly, since any sorting sequence for the SbT problem is a valid sorting sequence for SbRT, we have that $c_t^\lambda(\pi) \geq c_{rt}^\lambda(\pi) \geq 2\mathrm{Inv}(\pi)/(\lambda-1)$. However, we can achieve

a better result using the fact that $\Delta\text{Inv}(\pi, \tau)/|\tau| \leq \lambda/4$ for any λ-transposition τ (Lemma 3), which means that the minimum cost to decrease $\text{Inv}(\pi)$ by 1 is $4/\lambda$ when using λ-transpositions. Since any sorting sequence needs to decrease $\text{Inv}(\pi)$ inversions in π to sort the permutation, we have that $c_t^\lambda(\pi) \geq 4\text{Inv}(\pi)/\lambda$. □

Lemma 5 *For any permutation π and value $\lambda > 1$, $c_t^\lambda(\pi) \leq 2\,\text{Inv}(\pi)$ and $c_{rt}^\lambda(\pi) \leq c_r^\lambda(\pi) \leq 2\,\text{Inv}(\pi)$.*

Proof We need to prove that it is always possible to decrease the number of inversions by 1 with a cost less than or equal to 2, considering the three rearrangement models. If $\text{Inv}(\pi) = 0$, then π is already sorted. If $\text{Inv}(\pi) > 0$, then there exists an inversion (π_i, π_{i+1}) in π (Lemma 1). So, the transposition $\tau(i, i+1, i+2)$ and the reversal $\rho(i, i+1)$ both have cost 2 and $\Delta\text{Inv}(\pi, \tau(i, i+1, i+2)) = \Delta\text{Inv}(\pi, \rho(i, i+1)) = 1$, since the inversion (π_i, π_{i+1}) is removed. □

These algorithms generate sorting sequences, since they can always decrease the number of inversions in the permutation (Lemma 5) and $\text{Inv}(\pi) = 0$ if, and only if, π is equal to the identity, when considering unsigned permutations. From Lemmas 4 and 5, we obtain the result from Theorem 1.

Theorem 1 *Given an integer $\lambda > 1$, the inversion based greedy algorithms are $(\lambda-1)$-approximations for the SbR and SbRT problems and a $(\lambda/2)$-approximation for the SbT problem.*

4 Entropy based algorithms for unsigned permutations

We present λ-approximations for the problems SbR, SbT, and SbRT that use the concept of entropy. Before introducing the algorithms, we present how the rearrangements affect the entropy of a permutation and how we can always find a sequence of rearrangements that decreases the entropy of a permutation. The next lemmas present lower bounds for the sorting distance of π relating it to the value of $\text{ent}(\pi)$.

Lemma 6 *Given a permutation π and a λ-rearrangement β, we have that $\Delta\text{ent}(\pi, \beta)/|\beta| \leq \lambda/2$.*

Proof Consider that β is a λ-reversal of length k that affects the segment $\pi_i, \pi_{i+1}, \ldots, \pi_j$ transforming it into $\pi_j, \pi_{j-i}, \ldots, \pi_i$. For any element in this segment, the change in its entropy is at most the distance between its position in π and $\pi \cdot \beta$. For any $1 \leq \ell \leq \lfloor k/2 \rfloor$, the element $\pi_{i+\ell-1}$ was moved $k - (2\ell - 1)$ positions. In a similar way, for any $1 \leq \ell \leq \lfloor k/2 \rfloor$, the element $\pi_{j-\ell+1}$ was moved $k - (2\ell - 1)$ positions.

If k is even, then

$$\Delta\text{ent}(\pi, \beta) = \sum_{\ell=1}^{n} \Delta\text{ent}(\pi_\ell, \beta) = \sum_{\ell=i}^{j} \Delta\text{ent}(\pi_\ell, \beta),$$

since only the entropy of the affected elements changes. So,

$$\Delta\text{ent}(\pi,\beta) = \sum_{\ell=i}^{j} \Delta\text{ent}(\pi_\ell,\beta) = \sum_{\ell=1}^{\lfloor k/2\rfloor} \Delta\text{ent}(\pi_{i+\ell-1},\beta) + \sum_{\ell=1}^{\lfloor k/2\rfloor} \Delta\text{ent}(\pi_{j-\ell+1},\beta)$$
$$\leq 2\sum_{\ell=1}^{\lfloor k/2\rfloor}(k-(2\ell-1)) = 2\lceil k/2\rceil\lfloor k/2\rfloor.$$

If k is odd, then

$$\Delta\text{ent}(\pi,\beta) = \text{ent}(\pi_{i+\lfloor k/2\rfloor},\beta) + \sum_{\ell=1}^{\lfloor k/2\rfloor} \Delta\text{ent}(\pi_{i+\ell-1},\beta) + \sum_{\ell=1}^{\lfloor k/2\rfloor} \Delta\text{ent}(\pi_{j-\ell+1},\beta).$$

Since the element at position $i+\lfloor k/2\rfloor$ is not moved by the reversal, we have that $\text{ent}(\pi_{i+\lfloor k/2\rfloor}) = 0$. Therefore, the change in the entropy is at most $2\lceil k/2\rceil\lfloor k/2\rfloor \leq k^2/2$ and $\Delta\text{ent}(\pi,\beta)/|\beta| \leq k/2$. Note that this bound is maximum when $k=\lambda$, resulting in $\Delta\text{ent}(\pi,\beta)/|\beta| \leq \lambda/2$.

Now we consider that β is a λ-transposition $\tau(i,j,k)$ with $|\tau| = k-i = k'$. Consider applying the transposition that exchanges the segments $\pi_i,\ldots,\pi_{j-1}$ and $\pi_j,\ldots,\pi_{k-1}$. For the first segment, each element π_ℓ was moved $k-j$ positions, with $i \leq \ell < j$. And for the second segment, each element π_ℓ was moved $j-i$ positions, with $j \leq \ell < k$. Therefore,

$$\begin{aligned}\Delta\text{ent}(\pi,\beta) &= \sum_{\ell=i}^{j-1} \Delta\text{ent}(\pi_\ell,\beta) + \sum_{\ell=j}^{k-1} \Delta\text{ent}(\pi_\ell,\beta)\\ &\leq \sum_{\ell=i}^{j-1}(k-j) + \sum_{\ell=j}^{k-1}(j-i)\\ &= 2(k-j)(j-i) \leq 2(k'/2)^2, \text{ since } (k-j)+(j-i) = k'.\end{aligned}$$

Therefore, $\Delta\text{ent}(\pi,\beta)/|\beta| \leq k'/2$. As before, this bound is maximum when $k'=\lambda$, resulting in $\Delta\text{ent}(\pi,\beta)/|\beta| \leq \lambda/2$. □

Lemma 7 *For any permutation π and integer $\lambda > 1$, $c_r^\lambda(\pi) \geq c_{rt}^\lambda(\pi) \geq 2\text{ent}(\pi)/\lambda$ and $c_t^\lambda(\pi) \geq 2\text{ent}(\pi)/\lambda$.*

Proof By Lemma 4, we already have that $c_r^\lambda(\pi) \geq c_{rt}^\lambda(\pi)$. By Lemma 6, for any λ-rearrangement β, the ratio $\Delta\text{ent}(\pi,\beta)/|\beta|$ is at most $\lambda/2$. So, the minimum cost to decrease $\text{ent}(\pi)$ by 1 is $2/\lambda$. Any sorting sequence must decrease the entropy to 0, as $\text{ent}(\iota) = 0$, which results in the lower bound $2\text{ent}(\pi)/\lambda$ for both $c_{rt}^\lambda(\pi)$ and $c_t^\lambda(\pi)$. □

The next lemmas show how to find a sequence of rearrangements that decreases the entropy of a permutation and they give bounds for the cost of this sequence.

Lemma 8 *For any permutation π with* $\text{ent}(\pi) > 0$*, there exists a pair of elements π_i and π_j, with $1 \leq i < j \leq n$, such that $|\pi_i| \geq j$ and $|\pi_j| \leq i$.*

Proof Since $\text{ent}(\pi) > 0$, there exists an element out-of-place (i.e. $|\pi_k| \neq k$). We claim that there exists an element π_i with $|\pi_i| > i$. Let π_i be the element out-of-place with maximum absolute value $|\pi_i|$. Suppose, for the purpose of contradiction, that $|\pi_i| < i$. The element π_k, such that $|\pi_k| = i$, has absolute value greater than $|\pi_i|$ and is out-of-place, which contradicts our choice of π_i. Therefore, we have that $|\pi_i| > i$.

Now, let π_i be the rightmost element (i.e. i is maximum) such that $|\pi_i| > i$. Such an element must exist based on our previous claim. The element at position $k = |\pi_i|$ is out-of-place and we have that $|\pi_k| < k$, since $|\pi_i| = k > i$ and because of our choice of π_i. Now, let π_j be the leftmost (i.e. j is minimum) element in the segment $\pi_{i+1}, \ldots, \pi_k$ such that $|\pi_j| < j$. First, we have that $|\pi_i| = k \geq j$ and $i < j$. Therefore, we just need to prove that $|\pi_j| \leq i$. Suppose, for the purpose of contradiction, that $|\pi_j| > i$. Now, consider the element π_ℓ with $\ell = |\pi_j|$, which is out-of-place. We know that $i < \ell = |\pi_j| < j \leq k$ and either $|\pi_\ell| > \ell$ or $|\pi_\ell| < \ell$. If $|\pi_\ell| > \ell$, then this contradicts our choice of π_i. Otherwise, we have that $|\pi_\ell| < \ell$, which contradicts our choice of π_j. Since we reach a contradiction in both cases, we conclude that $|\pi_j| \leq i$. □

Lemma 9 *Given an unsigned permutation π with* $\text{ent}(\pi) > 0$*, there exists a sequence of super short rearrangements $S = \langle \beta_1, \beta_2, \ldots, \beta_m \rangle$ with $\Delta\text{ent}(\pi, S)/f(S) \geq 1/2$.*

Proof Let π_i and π_j be two elements of π, with $1 \leq i < j \leq n$, such that $|\pi_i| \geq j$ and $|\pi_j| \leq i$ (Lemma 8). Let $\pi' = (\pi_1 \ \ldots \ \pi_{i-1} \ \pi_j \ \pi_{i+1} \ \ldots \ \pi_{j-1} \ \pi_i \ \pi_{j+1} \ \ldots \ \pi_n)$. We have that $\text{ent}(\pi') = \text{ent}(\pi) - 2(j - i)$, since the entropy of the elements π_i and π_j decrease by $j - i$ each and the entropy remains the same for the other elements.

To exchange these two elements, we can use the sequence $\langle \rho(i, i+1), \rho(i+1, i+2), \ldots, \rho(j-1, j), \rho(j-2, j-1), \ldots, \rho(i+1, i) \rangle$, which has a cost less than or equal to $4(j - i)$, as it has length $(j - i) + (j - i - 1)$ and uses only super short reversals. Therefore,

$$\frac{\Delta\text{ent}(\pi, S)}{f(S)} \geq \frac{2(j-i)}{4(j-i)} = \frac{1}{2}.$$

The proof is similar for transpositions. □

Algorithm 2 describes the general idea for the entropy based greedy algorithms, whose greedy choice is to apply the operation that most decreases the entropy with the lowest cost, but uses Lemma 9 to guarantee that the sorting sequence S has $\Delta\text{ent}(\pi, S)/f(S) \geq 1/2$. The algorithm generates a sorting sequence, since it always decreases the entropy (Lemma 9) and $\text{ent}(\pi) = 0$ if, and only if, π is equal to the identity, when considering unsigned permutations.

Consider the unsigned permutation $\pi = (4\,5\,2\,3\,1)$, $\lambda = 3$, and the model containing λ-transpositions. We have $\text{ent}(\pi_1) = 3$, $\text{ent}(\pi_2) = 3$, $\text{ent}(\pi_3) = 1$, $\text{ent}(\pi_4) = 1$, $\text{ent}(\pi_5) = 4$, and $\text{ent}(\pi) = 12$. The algorithm applies the following sequence of λ-transpositions over π, which has a cost of 15:

$$\pi^0 = (4\ 5\ 2\ 3\ 1) = \pi$$

Algorithm 2: Entropy based greedy algorithms.

Input: An unsigned permutation π, a value $\lambda > 1$, and a model $\mathcal{M}$ containing only λ-rearrangements.
Output: A sequence of rearrangements from $\mathcal{M}$ that sorts π.

```
Let S be an empty sequence
while π is not equal to ι do
    Let β be the operation from M such that the ratio Δent(π, β)/f(β) is maximum
    if Δent(π, β)/f(β) ≥ 1/2 then
        Apply β in π
        Append β to S
    else
        Find π_i and π_j, with 1 ≤ i < j ≤ n, such that |π_i| ≥ j and |π_j| ≤ i
        Let S′ be the sequence from Lemma 9 that exchanges π_i and π_j
        Apply S′ in π
        Append S′ to S
return S
```

$$\pi^1 = \pi^0 \cdot \tau(2,3,5) = (4\ 2\ 3\ 5\ 1)$$
$$\pi^2 = \pi^1 \cdot \tau(4,5,6) = (4\ 2\ 3\ 1\ 5)$$
$$\pi^3 = \pi^2 \cdot \tau(1,2,3) \cdot \tau(2,3,4) \cdot \tau(3,4,5) \cdot \tau(2,3,4) \cdot \tau(1,2,3) = \iota$$

In this example, at the beginning of the last iteration, the permutation is equal to $\pi^2 = (4\ 2\ 3\ 1\ 5)$ and there are no λ-transpositions that decrease the entropy of the permutation. In this case, the algorithm exchanges the elements $\pi^2_1 = 4$ and $\pi^2_4 = 1$ with the sequence $S' = \langle \tau(1,2,3), \tau(2,3,4), \tau(3,4,5), \tau(2,3,4), \tau(1,2,3) \rangle$, such that $\Delta\text{ent}(\pi^2, S')/f(S') = 6/10 \geq 1/2$.

Theorem 2 *Given an integer $\lambda > 1$, the entropy based greedy algorithms are λ-approximation algorithms for SbR, SbT, and SbRT.*

Proof Consider the problem SbR and let S be the sorting sequence generated by the algorithm. By Lemma 9, the sequence S has $\Delta\text{ent}(\pi, S)/f(S) \geq 1/2$, that is, the average cost to decrease the entropy of π by 1 is less than or equal to 2. To sort the permutation, the sorting sequence S has to decrease $\text{ent}(\pi)$ units of entropy and, therefore, $f(S) \leq 2\text{ent}(\pi)$. From Lemma 7, we conclude that this algorithm is a λ-approximation.

The proof is similar for SbT and SbRT. □

5 Approximation algorithms for signed permutations

To show how these greedy algorithms work, we define a score function ϕ based on $\text{Inv}(\pi)$ and on the sets $E_\pi^{\text{even}^-}$ and $E_\pi^{\text{odd}^+}$.

Algorithm 3: Greedy algorithms that use the score function ϕ.

Input: A signed permutation π, a value $\lambda > 1$, and a model $\mathcal{M}$ containing only λ-rearrangements.
Output: A sequence of rearrangements from $\mathcal{M}$ that sorts π.

```
Let S be an empty sequence
while π is not equal to ι do
    Let β be the operation from M such that φ(π, β) is maximum
    Apply β in π
    Append β to S
return S
```

Given a permutation π and a rearrangement sequence S, the score function is defined as

$$\phi(\pi, S) = \frac{(2\,\mathrm{Inv}(\pi) + |E_\pi^{\mathrm{even}^-}| + |E_\pi^{\mathrm{odd}^+}|) - (2\,\mathrm{Inv}(\pi \cdot S) + |E_{\pi\cdot S}^{\mathrm{even}^-}| + |E_{\pi\cdot S}^{\mathrm{odd}^+}|)}{f(S)}.$$

The identity permutation is the only permutation with $\mathrm{Inv}(\pi) = |E_\pi^{\mathrm{even}^-}| = |E_\pi^{\mathrm{odd}^+}| = 0$. An optimal sorting sequence is the one with the highest score function. So, the greedy choice is to always apply the λ-rearrangement with the highest score function until the permutation is sorted. Algorithm 3 describes the general idea of these algorithms, which are used for the problems Sb$\bar{\mathrm{R}}$ and Sb$\bar{\mathrm{R}}$T.

Consider the signed permutation $\pi = (+3\ +4\ -5\ -2\ -1)$, $\lambda = 3$, and the model containing signed λ-reversals. This permutation has 7 inversions, $E_\pi^{\mathrm{even}^-} = \{-5, -2, -1\}$, and $E_\pi^{\mathrm{odd}^+} = \emptyset$. The algorithm applies the following sequence of signed λ-reversals over π, which has a cost of 11:

$$\begin{aligned}
\pi^0 &= (+3\ +4\ -5\ -2\ -1) = \pi \\
\pi^1 &= \pi^0 \cdot \bar{\rho}(3,5) = (+3\ +4\ +1\ +2\ +5) \\
\pi^2 &= \pi^1 \cdot \bar{\rho}(2,3) = (+3\ -1\ -4\ +2\ +5) \\
\pi^3 &= \pi^2 \cdot \bar{\rho}(1,2) = (+1\ -3\ -4\ +2\ +5) \\
\pi^4 &= \pi^3 \cdot \bar{\rho}(3,4) = (+1\ -3\ -2\ +4\ +5) \\
\pi^5 &= \pi^4 \cdot \bar{\rho}(2,3) = \iota
\end{aligned}$$

The following lemmas present a lower bound for the score function and show that it is always possible to find a signed λ-reversal with score function greater than or equal to 1.

Lemma 10 *For any signed permutation π and signed λ-reversal $\bar{\rho}$, we have that $\phi(\pi, \bar{\rho}) \leq \lambda$.*

Proof Let $\pi' = \pi \cdot \bar{\rho}$ and suppose that $|\bar{\rho}| = k \leq \lambda$. By Lemma 3, we have that $2\,\mathrm{Inv}(\pi) - 2\,\mathrm{Inv}'(\pi) \leq k(k-1)$. Moreover, since k elements are affected by $\bar{\rho}$ and $E_\pi^{\mathrm{even}^-} \cap E_\pi^{\mathrm{odd}^+} = \emptyset$, we have that $(|E_\pi^{\mathrm{even}^-}| + |E_\pi^{\mathrm{odd}^+}|) - (|E_{\pi'}^{\mathrm{even}^-}| + |E_{\pi'}^{\mathrm{odd}^+}|) \leq k$.

Combining these two inequalities results in

$$(2\,\mathrm{Inv}(\pi) + |E_\pi^{\mathrm{even}^-}| + |E_\pi^{\mathrm{odd}^+}|) - (2\,\mathrm{Inv}(\pi') + |E_{\pi'}^{\mathrm{even}^-}| + |E_{\pi'}^{\mathrm{odd}^+}|) \leq k^2,$$

which implies that

$$\frac{(2\,\mathrm{Inv}(\pi) + |E_\pi^{\mathrm{even}^-}| + |E_\pi^{\mathrm{odd}^+}|) - (2\,\mathrm{Inv}(\pi') + |E_{\pi'}^{\mathrm{even}^-}| + |E_{\pi'}^{\mathrm{odd}^+}|)}{|\bar{\rho}|} \leq \frac{k^2}{k} = k.$$

Since $1 \leq |\bar{\rho}| \leq \lambda$, we have that $\phi(\pi, \bar{\rho}) \leq \lambda$. □

Lemma 11 *For any signed permutation π and λ-transposition τ, we have that $\phi(\pi, \tau) \leq \lambda$.*

Proof Similar to the proof of Lemma 10. □

Lemma 12 *For any signed permutation $\pi \neq \iota$, there exists a super short signed reversal $\bar{\rho}$ with $\phi(\pi, \bar{\rho}) \geq 1$.*

Proof Let $\pi' = \pi \cdot \bar{\rho}$. If $\mathrm{Inv}(\pi) = 0$, then $\mathrm{ent}(\pi_i) = 0$ for all elements π_i of π and $|E_\pi^{\mathrm{odd}^+}| = 0$. As $\pi \neq \iota$, there exists an element $\pi_i < 0$ in $E_\pi^{\mathrm{even}^-}$ and, after applying $\bar{\rho}(i, i)$ on π, this element becomes positive in π'. Therefore, we have that $|E_\pi^{\mathrm{even}^-}| - |E_{\pi'}^{\mathrm{even}^-}| = 1$ and $\phi(\pi, \bar{\rho}) = 1$.

If $\mathrm{Inv}(\pi) > 0$, then there exists an inversion (π_i, π_{i+1}) in π according to Lemma 1. Thus, the signed reversal $\bar{\rho}(i, i+1)$ removes this inversion, resulting in $2(\Delta\mathrm{Inv}(\pi, \bar{\rho})) = 2$ and $|E_\pi^{\mathrm{even}^-}| + |E_\pi^{\mathrm{odd}^+}| = |E_{\pi'}^{\mathrm{even}^-}| + |E_{\pi'}^{\mathrm{odd}^+}|$ (Lemma 2). Therefore, $\phi(\pi, \bar{\rho}) = 1$. □

From Lemmas 10, 11, and 12, we obtain the result from Theorem 3.

Theorem 3 *Given an integer $\lambda > 1$, Algorithm 3 is a λ-approximation for the problems $Sb\bar{\mathrm{R}}$ and $Sb\bar{\mathrm{R}}T$.*

6 Algorithms for large values of λ

This section shows algorithms for large values of λ, that is, $\lambda = \Theta(n)$. For this, we present the relation between the problems of Sorting by Length-Weighted λ-Rearrangements and the problems of Sorting by Length-Weighted Rearrangements. Observe that the problem without the restriction in the rearrangement's length is equivalent to set $\lambda = n$.

Lemma 13 *Given a permutation π, we have that $c_M^\lambda(\pi) \geq c_M^n(\pi)$ for $M \in \{r, \bar{r}, t, rt, \bar{r}t\}$.*

Proof Consider the problem with unsigned reversals (for the other models the proof is similar). An optimal sorting sequence, with cost $c_M^\lambda(\pi)$, for the problem of Sorting by Length-Weighted λ-Reversals is also a valid sorting sequence for the problem of

Sorting by Length-Weighted Reversals. Therefore, we conclude that an optimal sorting sequence for the Sorting by Length-Weighted Reversals has cost less than or equal to $c_M^{\lambda}(\pi)$. □

Lemma 14 *Given a permutation π and an integer $\lambda > 1$, for any rearrangement β, with $|\beta| > \lambda$, there exists a sequence S with only λ-rearrangements such that $\pi \cdot \beta = \pi \cdot S$ and $f(S) = O(|\beta|^2/\lambda)$.*

Proof Consider that β is a (signed or unsigned) reversal $\rho(i, j)$ with length $j-i+1 > \lambda$. Miranda et al. (2019) showed that the effect of this reversal can be obtained using a sequence S with at most $q(q+1)/2$ λ-reversals, where $q = \lceil (j-i+1)/\lfloor \lambda/2 \rfloor \rceil$. They also showed that S has at most $q(q-1)/2$ reversals of length λ and at most q reversals of length $\lfloor \lambda/2 \rfloor$. Therefore,

$$f(S) \leq \lambda \frac{q(q-1)}{2} + \lfloor \lambda/2 \rfloor q \leq \lambda \frac{q^2}{2} \leq \lambda/2 \lceil \frac{j-i+1}{\lfloor \lambda/2 \rfloor} \rceil^2 \leq \lambda/2 \lceil \frac{|\beta|}{\lfloor \lambda/2 \rfloor} \rceil^2,$$

which is $O(|\beta|^2/\lambda)$.

Now, consider that β is a transposition $\tau(i, j, k)$ with length $k - i > \lambda$. Miranda et al. (2019) also showed that the effect of this transposition can be obtained using a sequence S with at most $\lceil (j-i)/\lfloor \lambda/2 \rfloor \rceil \lceil (k-j)/\lfloor \lambda/2 \rfloor \rceil$ λ-transpositions. Since $k - j < k - i$ and $j - i < k - i$, we have that

$$f(S) \leq \lambda \lceil \frac{j-i}{\lfloor \lambda/2 \rfloor} \rceil \lceil \frac{k-j}{\lfloor \lambda/2 \rfloor} \rceil \leq \lambda \lceil \frac{|\beta|}{\lfloor \lambda/2 \rfloor} \rceil^2,$$

which is also $O(|\beta|^2/\lambda)$. □

Using Lemma 14, we can adapt approximation algorithms from the Sorting by Length-Weighted Rearrangements by adding the restriction that all rearrangements in the sorting sequence are λ-rearrangements. Next lemmas shows how this procedure changes the approximation factor of these algorithms.

Theorem 4 *An x-approximation algorithm for the Sorting by Length-Weighted (Signed or Unsigned) Reversals can be adapted to an $O(x\,n/\lambda)$-approximation algorithm for the Sorting by Length-Weighted λ-Reversals.*

Proof Given an unsigned permutation π, the x-approximation algorithm returns a sorting sequence S with cost $f(S) \leq x c_r^n(\pi)$. Applying Lemma 14, we construct a valid sorting sequence S' containing only λ-reversals, such that

$$\begin{aligned} f(S') &\leq \sum_{\rho \in S} \frac{c|\rho|^2}{\lambda}, \text{ for some constant } c \\ &\leq \sum_{\rho \in S} \frac{cn|\rho|}{\lambda} = \frac{cn}{\lambda} \sum_{\rho \in S} |\rho| \\ &= \frac{cn}{\lambda} f(S) \leq \frac{cn}{\lambda} x c_r^n(\pi). \end{aligned}$$

By Lemma 13, this adapted algorithm is an $O(x\,n/\lambda)$-approximation algorithm for the Sorting by Length-Weighted λ-Reversals. The same is valid for signed reversals. □

We can obtain Theorems 5 and 6 with a proof similar to the one used for Theorem 4.

Theorem 5 *An x-approximation algorithm for the Sorting by Length-Weighted Transpositions can be adapted to an $O(x\,n/\lambda)$-approximation algorithm for the Sorting by Length-Weighted λ-Transpositions.*

Theorem 6 *An x-approximation algorithm for the Sorting by Length-Weighted (Signed or Unsigned) Reversals and Transpositions can be adapted to an $O(x\,n/\lambda)$-approximation algorithm for the Sorting by Length-Weighted λ-Reversals and λ-Transpositions.*

From Theorem 4, we have $O(n \lg n/\lambda)$-approximation algorithms for SbR and Sb$\bar{\text{R}}$, adapting algorithms presented by Bender et al. (2008) and Swidan et al. (2004), respectively. Also, from Theorems 5 and 6, we have $O(n \lg^2 n/\lambda)$-approximation algorithms for SbT, SbRT, and Sb$\bar{\text{R}}$T, adapting algorithms from Lintzmayer et al. (2018). For $\lambda = \Theta(n)$, these algorithms present better approximation factors since n/λ is bounded by some constant and, therefore, these adapted algorithms have approximation factors of $O(\lg n)$ or $O(\lg^2 n)$. In contrast, the algorithms presented in the previous sections have an approximation factor of $O(n)$ for large values of λ.

7 Length-weighted short rearrangement distance

One well studied variant of the rearrangement problems is the one considering only short rearrangements. The problems of Sorting by Super Short Rearrangements (Reversals or Transpositions) are polynomial (Jerrum 1985; Galvão et al. 2015). The problems of Sorting Signed and Unsigned Permutations by Short Reversals have approximation algorithms with factors 5 (Galvão et al. 2015) and 2 (Heath and Vergara 2003), respectively. The best-known algorithm for the problem of Sorting (Unsigned) Permutations by Short Transpositions has approximation factor of 5/4 (Jiang et al. 2014). For this problem, there also exists a $(1 + n/\text{Inv}(\pi))$-approximation (Jiang et al. 2012), where n is the number of elements in the permutation. For the problems of Sorting Signed and Unsigned Permutations by Short Reversals and Short Transpositions, there exist algorithms with approximation factors of 3 (Galvão et al. 2015) and 2 (Vergara 1998), respectively.

In this section, we analyze the approximation algorithms presented in Sects. 3–5 when considering only short rearrangements, that is, when $\lambda = 3$. For the problems SbR and SbRT, the inversions based greedy algorithms are 2-approximations when $\lambda = 3$. For the problem SbT, the approximation factor becomes 1.5, however we show that this algorithm is in fact a 4/3-approximation in the next lemma.

Lemma 15 *The inversions based greedy algorithm is a 4/3-approximation for the problem of Sorting Permutations by Length-Weighted Short Rearrangements.*

Proof Using a similar argument to the one used in Lemma 3, for a short transposition $\tau(i, j, k)$ with length k' we have

$$\frac{\Delta\mathrm{Inv}(\pi, \tau)}{|\tau|} \leq \frac{(j-i)(k-j)}{k'} \leq \frac{2}{3}, \text{ since } (j-i)+(k-j) = k' \leq 3.$$

Therefore, $c_t^3(\pi) \geq 3\mathrm{Inv}(\pi)/2$, for any permutation π. Since the algorithm uses a sorting sequence with cost less than or equal to $2\mathrm{Inv}(\pi)$, we conclude that its approximation factor is $2\mathrm{Inv}(\pi)/(3/2\mathrm{Inv}(\pi)) = 4/3$. □

Considering signed permutations, the algorithms presented in Sect. 5 are 3-approximations for Sb$\bar{\mathrm{R}}$ and Sb$\bar{\mathrm{R}}$T when $\lambda = 3$. Now, we present a 7/3-approximation algorithm using a new score function based on the inversion graph of a permutation π.

Given a permutation π and a rearrangement sequence S, the score function ψ is defined as

$$\psi(\pi, S) = \frac{(2\,\mathrm{Inv}(\pi) + c_{\mathrm{odd}}(\pi)) - (2\,\mathrm{Inv}(\pi \cdot S) + c_{\mathrm{odd}}(\pi \cdot S))}{f(S)}.$$

The identity permutation is the only one with $\mathrm{Inv}(\pi) + c_{\mathrm{odd}}(\pi) = 0$. The greedy choice of the algorithm is to always apply the short rearrangement with the highest value of ψ until the permutation is equal to the identity. Next lemmas present bounds on the values of the score function.

Lemma 16 (Alexandrino et al. 2019) *For any signed permutation π and short rearrangement β, we have that $\psi(\pi, \beta) \leq 7/3$.*

Lemma 17 (Alexandrino et al. 2019) *For any signed permutation $\pi \neq \iota$, there exists a short rearrangement β with $\psi(\pi, \beta) \geq 1$.*

Theorem 7 (Alexandrino et al. 2019) *The algorithm based on the score function ψ is a 7/3-approximation algorithm for Sb$\bar{\mathrm{R}}$T when $\lambda = 3$.*

8 Approximation factors for $\alpha > 1$

In the previous sections, we considered that $\alpha = 1$, that is, $f(\beta) = |\beta|^\alpha = |\beta|$ for any rearrangement β. In this section, we analyze how the approximation factors of the algorithms presented change as the value of α increases. The main idea is to show that, as the value of α increases, the cost-benefit of super short rearrangements becomes better, which improves the approximation factor of our algorithms presented in Sects. 3 and 5. We begin by analyzing the problems involving unsigned permutations and presenting new lower bounds and upper bounds for these problems.

Lemma 18 *Given an unsigned permutation π, an integer $\lambda > 1$, and a real value $\alpha > 1$, any λ-rearrangement β has ratio*

$$\frac{\Delta\mathrm{Inv}(\pi, \beta)}{|\beta|^\alpha} \leq \max_{2 \leq k \leq \lambda} \left\{ \frac{(k-1)}{2k^{\alpha-1}} \right\}.$$

Proof Similar to the proof of Lemma 3. □

Lemma 19 *For any permutation π, integer $\lambda > 1$, and real value $\alpha > 1$, we have that $c_r^\lambda(\pi) \geq \mathrm{Inv}(\pi)/t$, $c_{rt}^\lambda(\pi) \geq \mathrm{Inv}(\pi)/t$, and $c_t^\lambda(\pi) \geq \mathrm{Inv}(\pi)/t$, where $t = \max_{2\leq k\leq\lambda}\left\{\frac{(k-1)}{2k^{\alpha-1}}\right\}$.*

Proof Similar to the proof of Lemma 4, using Lemma 18. □

Lemma 20 *Given a real value $\alpha > 1$, the inversions based greedy algorithms presented in Sect. 3 sort an unsigned permutation π with cost less than or equal to $2^\alpha \mathrm{Inv}(\pi)$.*

Proof Similar to the proof of Lemma 5. □

Theorem 8 *For $\alpha \geq 2$ and an integer $\lambda > 1$, there exists a 2-approximation algorithm for the problems SbR, SbT, and SbRT.*

Proof For $\alpha \geq 2$, we have that

$$\max_{2\leq k\leq\lambda}\left\{\frac{(k-1)}{2k^{\alpha-1}}\right\} < \max_{2\leq k\leq\lambda}\left\{\frac{1}{2k^{\alpha-2}}\right\} \leq \frac{1}{2^{\alpha-1}}.$$

Note that, since $\alpha \geq 2$, the expression above reaches its maximum value when $k = 2$.

Considering the problem SbR, $c_r^\lambda(\pi) \geq 2^{\alpha-1}\mathrm{Inv}(\pi)$ according to Lemma 19. By Lemma 20, the algorithm uses a sorting sequence of cost less than or equal to $2^\alpha \mathrm{Inv}(\pi)$. Therefore, the approximation factor of this algorithm is

$$\frac{2^\alpha \mathrm{Inv}(\pi)}{2^{\alpha-1}\mathrm{Inv}(\pi)} = 2.$$

The proof is similar considering SbT and SbRT. □

Theorem 9 *For $\alpha \geq 3$, the problems SbR, SbT, and SbRT can be solved in polynomial time.*

Proof Consider the problem SbR. For any $k \geq 4$ and $\alpha \geq 3$, we have that

$$\frac{(k-1)}{2k^{\alpha-1}} \leq \frac{1}{2k^{\alpha-2}} \leq \frac{1}{2^\alpha}, \text{ since } k \geq 4.$$

Moreover, $(k-1)/(2k^{\alpha-1}) = 1/(9k^{\alpha-3}) \leq 1/2^\alpha$ for $k = 3$, and $(k-1)/(2k^{\alpha-1}) = 1/2^\alpha$ for $k = 2$. Therefore, $\max_{2\leq k\leq\lambda}\left\{(k-1)/(2k^{\alpha-1})\right\} = 1/2^\alpha$ and $c_r^\lambda(\pi) \geq 2^\alpha \mathrm{Inv}(\pi)$, using Lemma 19.

By Lemma 20, the algorithm uses a sorting sequence of cost less than or equal to $2^\alpha \mathrm{Inv}(\pi)$ and, therefore, this sorting sequence is optimal. The proof is similar considering SbT and SbRT. □

In the following lemmas, we present bounds on the value of the score function ϕ.

Lemma 21 *Given a signed permutation π, an integer $\lambda > 1$, and a real value $\alpha > 1$, any λ-rearrangement β has $\phi(\pi, \beta) \leq \max_{1 \leq k \leq \lambda}\{k^2/k^\alpha\}$.*

Proof Similar to the proofs of Lemmas 10 and 11. □

Corollary 1 *Given a signed permutation π, an integer $\lambda > 1$, and a real value $\alpha \geq 2$, any λ-rearrangement β has $\phi(\pi, \beta) \leq 1$.*

Lemma 22 *Given a value $\alpha > 1$, the algorithms presented in Sect. 5 sort a signed permutation π with a sequence S such that $\phi(\pi, S) \geq 1/2^\alpha$.*

Proof Similar to the proof of Lemma 12. □

From Corollary 1 and Lemma 22, we obtain the result from Theorem 10.

Theorem 10 *For $\alpha \geq 2$, there exists a 2^α-approximation algorithm for the problems Sb$\bar{\text{R}}$ and Sb$\bar{\text{R}}$T.*

In the following lemmas, we show how to obtain an approximation factor of 3 for the problems involving signed permutations using new lower bounds.

Theorem 11 *For $\alpha \geq 2$, there exists 3-approximation algorithms for the problems Sb$\bar{\text{R}}$ and Sb$\bar{\text{R}}$T.*

Proof Recall that the identity permutation is the only signed permutation with $\text{Inv}(\pi) = |E_\pi^{\text{even}^-}| = |E_\pi^{\text{odd}^+}| = 0$. Since a reversal of length 1 does not change the number of inversions in a permutation and using an argument similar to the one of Lemma 8, we have that $c_{\bar{r}}^\lambda(\pi) \geq c_{\bar{r}t}^\lambda(\pi) \geq 2^{\alpha-1}\text{Inv}(\pi)$.

A rearrangement of length k can reduce the size of $E_\pi^{\text{even}^-} \cup E_\pi^{\text{odd}^+}$ by at most k, since exactly k elements are affected and $E_\pi^{\text{even}^-} \cap E_\pi^{\text{odd}^+} = \emptyset$. As we are considering $\alpha \geq 2$ and any sorting sequence makes $E_\pi^{\text{even}^-} \cup E_\pi^{\text{odd}^+}$ empty, we have $c_{\bar{r}}^\lambda(\pi) \geq c_{\bar{r}t}^\lambda(\pi) \geq |E_\pi^{\text{even}^-}| + |E_\pi^{\text{odd}^+}|$.

Consider only the problem Sb$\bar{\text{R}}$. Given a permutation π, we can sort this permutation with the following algorithm: (i) while $\text{Inv}(\pi) > 0$, apply a signed reversal of length 2 that removes an inversion of π (Lemma 1); (ii) when $\text{Inv}(\pi) = 0$, we have that $\text{ent}(\pi_i) = 0$ for all elements π_i of π and $|E_\pi^{\text{odd}^+}| = 0$, so we apply a signed reversal of length 1 in each element of $E_\pi^{\text{even}^-}$ to make this set empty. Note that the size of set $E_\pi^{\text{even}^-} \cup E_\pi^{\text{odd}^+}$ remains the same after the first step as we only use signed reversals of length 2 (Lemma 2). Moreover, in the last step, we do not increase the size of $E_\pi^{\text{odd}^+}$, which is already empty. So, this procedure generates a sorting sequence as it makes $\text{Inv}(\pi) = |E_\pi^{\text{even}^-}| = |E_\pi^{\text{odd}^+}| = 0$.

The first step uses a sequence with cost $2^\alpha\text{Inv}(\pi)$ and the second step uses a sequence with cost $|E_\pi^{\text{even}^-}| + |E_\pi^{\text{odd}^+}|$. So, the sorting sequence has cost

$$2^\alpha\text{Inv}(\pi) + |E_\pi^{\text{even}^-}| + |E_\pi^{\text{odd}^+}| \leq 2c_{\bar{r}}^\lambda(\pi) + c_{\bar{r}}^\lambda(\pi) = 3c_{\bar{r}}^\lambda(\pi).$$

We conclude that this algorithm is a 3-approximation for Sb$\bar{\text{R}}$. The proof is analogous for Sb$\bar{\text{R}}$T. □

Lemma 23 *For $\alpha \geq 3$ and any signed λ-reversal $\bar{\rho}$, there exists a sequence S such that $\pi \cdot \bar{\rho} = \pi \cdot S$ and $f(S) \leq |\bar{\rho}|^{\alpha}$, for any signed permutation π.*

Proof Let $\bar{\rho} = \bar{\rho}(i, j)$ and $\pi' = \pi \cdot \bar{\rho}(i, j)$. We construct a sequence that moves each element of π to its correct position in π' using only super short signed reversals going from i to j. After the elements are in the correct positions, we use another sequence of reversals of length 1 to fix the signs of the elements, if necessary. Next, we show an example of such sequence when $|\bar{\rho}| = 3$:

$$
\begin{aligned}
\pi^0 &= \pi & &= (\pi_1 \ \ldots \ \pi_{i-1} \ \pi_i \ \pi_{i+1} \ \pi_{i+2} \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^1 &= \pi^0 \cdot \bar{\rho}(i, i{+}1) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ \underline{-\pi_{i+1} \ -\pi_i} \ \pi_{i+2} \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^2 &= \pi^1 \cdot \bar{\rho}(i{+}1, i{+}2) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ -\pi_{i+1} \ \underline{-\pi_{i+2} \ \pi_i} \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^3 &= \pi^2 \cdot \bar{\rho}(i, i{+}1) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ \underline{\pi_{i+2} \ \pi_{i+1}} \ \pi_i \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^4 &= \pi^3 \cdot \bar{\rho}(i, i) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ \underline{-\pi_{i+2}} \ \pi_{i+1} \ \pi_i \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^5 &= \pi^4 \cdot \bar{\rho}(i{+}1, i{+}1) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ -\pi_{i+2} \ \underline{-\pi_{i+1}} \ \pi_i \ \pi_{i+3} \ \ldots \ \pi_n)\\
\pi^6 &= \pi^5 \cdot \bar{\rho}(i{+}2, i{+}2) & &= (\pi_1 \ \ldots \ \pi_{i-1} \ -\pi_{i+2} \ -\pi_{i+1} \ \underline{-\pi_i} \ \pi_{i+3} \ \ldots \ \pi_n)
\end{aligned}
$$

Let $k = |\bar{\rho}|$. We need to use $k - \ell$ reversals of length 2 to move the ℓ-th element in the subsequence $\pi_i, \ldots, \pi_j$ to its correct position in π'. After moving all elements, they may have an incorrect sign and, in order to fix this, we use at most k reversals of length 1. Therefore, the sequence has cost at most

$$2^{\alpha} \sum_{\ell=1}^{k} (k - \ell) + k = 2^{\alpha} \frac{k(k-1)}{2} + k = 2^{\alpha-1} k(k-1) + k.$$

For $k = 3$,

$$2^{\alpha-1} k(k-1) + k = 2^{\alpha} 3 + 3 \leq 3^{\alpha} = |\bar{\rho}|^{\alpha}, \text{ since } \alpha \geq 3.$$

For $k \geq 4$,

$$2^{\alpha-1} k(k-1) + k \leq 2^{\alpha-1} k^2 \leq k^{(\alpha-1)/2} k^2 \leq k^{\alpha} = |\bar{\rho}|^{\alpha}, \text{ since } \alpha \geq 3.$$

Therefore, the sequence has cost less than or equal to $|\bar{\rho}|^{\alpha}$. □

Theorem 12 *For $\alpha \geq 3$, the problem $Sb\bar{\mathrm{R}}$ can be solved in polynomial time for any integer $\lambda > 1$.*

Proof By Lemma 23, considering $\alpha \geq 3$, there exists an optimal sorting sequence for π using only super short signed reversals. Let S be such an optimal sorting sequence. This sorting sequence removes $\mathrm{Inv}(\pi)$ inversions and decreases the size of $E_{\pi}^{\text{even}^-} \bigcup E_{\pi}^{\text{odd}^+}$ by $|E_{\pi}^{\text{even}^-}| + |E_{\pi}^{\text{odd}^+}|$ units.

Since a signed reversal of length 2 does not change the size of $E_{\pi}^{\text{even}^-} \bigcup E_{\pi}^{\text{odd}^+}$ (Lemma 2) and a signed reversal of length 1 decreases the size of this set by at most 1 unit, we have that S has at least $|E_{\pi}^{\text{even}^-}| + |E_{\pi}^{\text{odd}^+}|$ signed reversals of length 1.

Besides that, a signed reversal of length 1 does not change the inversions of a permutation and a signed reversal of length 2 decreases the number of inversions of π by at most 1 unit. In this way, S has at least $\mathrm{Inv}(\pi)$ signed reversals of length 2. Therefore, $f(S) \geq 2^{\alpha}\mathrm{Inv}(\pi) + |E_{\pi}^{\mathrm{even}^-}| + |E_{\pi}^{\mathrm{odd}^+}|$.

Using an argument similar to the one used in Lemma 12, we can find a sorting sequence S' with cost equal to $2^{\alpha}\mathrm{Inv}(\pi) + |E_{\pi}^{\mathrm{even}^-}| + |E_{\pi}^{\mathrm{odd}^+}|$. From the lower bound presented above, we conclude that this sorting sequence is optimal. □

Lemma 24 *For $\alpha \geq 3$ and any λ-transposition τ, there exists a sequence S such that $\pi \cdot \tau = \pi \cdot S$ and $f(S) \leq |\tau|^{\alpha}$, for any signed permutation π.*

Proof Similar to the proof of Lemma 23. □

Lemma 25 *For any signed permutation π, if a signed reversal or transposition β with length 2 removes an inversion of π, then $c_{\mathrm{odd}}(\pi) \leq c_{\mathrm{odd}}(\pi \cdot \beta)$, that is, the number of odd components of the inversion graph does not decrease.*

Proof Consider that β is a signed reversal. Let π_i and π_{i+1} be the elements affected by this reversal. Since only these two elements are affected, the inversion removed by β is (π_i, π_{i+1}). Let $e = (\pi_i, \pi_{i+1})$ be the removed edge of $G(\pi)$ and C be the component containing e. We divide our analysis into two cases:

1. e is a cut-edge: let C_1 and C_2 be the components of $C - e$ in $G(\pi)$. Since e is a cut-edge, the elements π_i and π_{i+1} are in distinct components of $C - e$. If C is odd, then either C_1 or C_2 is odd. Suppose, without loss of generality, that C_1 is odd and C_2 is even. After applying the signed reversal, the component C_1 becomes even and the component C_2 becomes odd. In this way, the number of odd components remains the same. If C is even, then both C_1 and C_2 have the same parity. After applying the signed reversal, the number of odd components can only increase. In both cases, we have $c_{\mathrm{odd}}(\pi) \leq c_{\mathrm{odd}}(\pi \cdot \beta)$.
2. e is not a cut-edge: in this case, the component C has the same set of vertices and its parity remains the same. Therefore, $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$.

The proof is analogous for a transposition of length 2. □

Lemma 26 *For any signed permutation π, if $\mathrm{Inv}(\pi) > 0$, then there exists a rearrangement β of length 2 such that $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$.*

Proof Since $\mathrm{Inv}(\pi) > 0$, there exists an edge $e = (\pi_i, \pi_{i+1})$ in $G(\pi)$. Let C be the component that contains e. Suppose that e is not a cut-edge. A transposition $\beta = \tau(i, i+1, i+2)$ removes the inversion (π_i, π_{i+1}) and, since e is not a cut-edge, the component C has the same set of vertices and its parity remains the same, as a transposition does not change signs of elements. So, we have $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$.

Now, suppose that e is a cut-edge. Let C_1 and C_2 be the components of $C - e$. We further divide this analysis into the following cases:

1. C_1 and C_2 are both even. After applying $\beta = \tau(i, i+1, i+2)$ on π, we have that $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$. Note that C is also even and the parity of the components C_1 and C_2 remains the same.

2. C_1 and C_2 are both odd. After applying $\beta = \bar{\rho}(i, i+1)$ on π, we have that $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$. Note that C is even and the components C_1 and C_2 become even after applying the signed reversal in π.
3. C_1 and C_2 have distinct parities. After applying $\beta = \tau(i, i+1, i+2)$ on π, we have that $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$. Note that C is odd and the parity of the components C_1 and C_2 remains the same.

In all cases, we have that $\mathrm{Inv}(\pi \cdot \beta) = \mathrm{Inv}(\pi) - 1$ and $c_{\mathrm{odd}}(\pi) = c_{\mathrm{odd}}(\pi \cdot \beta)$. □

Lemma 27 *For any signed permutation π and real value $\alpha \geq 1$, we have $c^2_{rt}(\pi) \leq c_{\mathrm{odd}}(\pi) + 2^{\alpha}\mathrm{Inv}(\pi)$.*

Proof To sort π, we use the following algorithm:

1. While $\mathrm{Inv}(\pi) > 0$, find a transposition of length 2 which removes an inversion and that does not increase the number of odd components (Lemma 26).
2. For each odd component of $G(\pi)$, apply a signed reversal of length 1 to decrease the number of odd components by one. Note that if $\mathrm{Inv}(\pi) = 0$, then there is no edge in $G(\pi)$ and each component of $G(\pi)$ is a single vertex. Each odd component $C = \{\pi_i\}$ becomes even after applying $\beta = \bar{\rho}(i, i)$ in π.

Since the identity permutation is the only one with $\mathrm{Inv}(\pi) + c_{\mathrm{odd}}(\pi) = 0$, this algorithm sorts π. At the first step, the algorithm uses $\mathrm{Inv}(\pi)$ transpositions of length 2 and, at the second step, the algorithm uses $c_{\mathrm{odd}}(\pi)$ signed reversals of length 1. Therefore, the sorting sequence has cost $c_{\mathrm{odd}}(\pi) + 2^{\alpha}\mathrm{Inv}(\pi)$. □

Theorem 13 *For $\alpha \geq 3$, the problem $Sb\bar{\mathrm{R}}T$ can be solved in polynomial time for any integer $\lambda > 1$.*

Proof By Lemmas 23 and 24, considering $\alpha \geq 3$, there exists an optimal sorting sequence for π using only super short signed reversals and/or super short transpositions. Let S be such an optimal sorting sequence. This sequence removes $\mathrm{Inv}(\pi)$ inversions and decreases the number of odd components of $G(\pi)$ by $c_{\mathrm{odd}}(\pi)$ units.

A signed reversal of length 1 does not change the inversions of a permutation. Also, a rearrangement of length 2 can remove at most one inversion. Therefore, S has at least $\mathrm{Inv}(\pi)$ rearrangements of length 2. Let S_{inv} be this subset of rearrangements which remove inversions.

As shown by Lemma 25, the rearrangements of S_{inv} do not decrease the number of odd components of the inversion graph. Let S' be the subset of rearrangements $S \setminus S_{inv}$. The rearrangements of S' must remove at least $c_{\mathrm{odd}}(\pi)$ odd components of the inversion graph. A rearrangement of length 1 removes at most one odd component and a rearrangement of length 2 removes at most two odd components. In this way, the cost to remove one odd component is greater than or equal to 1, considering $\alpha \geq 2$. Therefore, the cost of S' is greater than or equal to $c_{\mathrm{odd}}(\pi)$ and $f(S) \geq 2^{\alpha}\mathrm{Inv}(\pi) + c_{\mathrm{odd}}(\pi)$.

Lemma 27 shows how to find a sorting sequence with cost $2^{\alpha}\mathrm{Inv}(\pi) + c_{\mathrm{odd}}(\pi)$. Using our previous lower bound, we conclude that this sorting sequence is optimal. □

Table 1 Summary of the results obtained for the Sorting by Length-Weighted λ-Rearrangements problems

Parameters	Best approximation factor				
	SbR	SbT	SbRT	Sb$\bar{\text{R}}$	Sb$\bar{\text{R}}$T
$\lambda > 1$ and $\alpha = 1$	$\lambda - 1$	$\lambda/2$	$\lambda - 1$	λ	λ
$\lambda = \Theta(n)$ and $\alpha = 1$	$O(\lg n)$	$O(\lg^2 n)$	$O(\lg^2 n)$	$O(\lg n)$	$O(\lg^2 n)$
$\lambda = 3$ and $\alpha = 1$	2	4/3	2	3	7/3
$\lambda > 1$ and $\alpha \geq 2$	2	2	2	3	3
$\lambda > 1$ and $\alpha \geq 3$	Exact	Exact	Exact	Exact	Exact

9 Conclusion

In this work, we presented $O(\lambda)$-approximation algorithms for the problems of Sorting by Length-Weighted λ-Rearrangements with $\alpha = 1$. These algorithms were based on the concepts of inversions and/or entropy. For large values of λ (i.e. $\lambda = \Theta(n)$), we were able to improve the approximation factor to $O(\lg n)$, for reversals, and $O(\lg^2 n)$, for transpositions and the combination of reversals and transpositions. Also, we presented algorithms with constant approximation factor for length-weighted short rearrangements, still considering $\alpha = 1$.

For $\alpha \geq 2$, we gave a 2-approximation algorithm for sorting unsigned permutations and a 3-approximation algorithm for sorting signed permutations. When $\alpha \geq 3$, we proved that, for each of the five rearrangements models studied in this work, the problem becomes solvable in polynomial time. The best approximation factors obtained for each problem are shown in Table 1.

For future work, we aim at improving the approximation factor when considering $\alpha = 1$ and any value of λ. Besides that, there are other variations of these problems to be considered which allow rearrangements such as insertions and deletions of genes. In this case, the genomes may not have the same set of genes.

Acknowledgements This work was supported by the National Counsel of Technological and Scientific Development, CNPq (Grants 400487/2016-0, 425340/2016-3, 131182/2017-0, and 304380/2018-0), São Paulo Research Foundation, FAPESP (Grants 2015/11937-9, 2017/12646-3, and 2017/16246-0), and by the Brazilian Federal Agency for the Support and Evaluation of Graduate Education, CAPES (Finance Code 001).

References

Alexandrino AO, Lintzmayer CN, Dias Z (2018) Approximation algorithms for sorting permutations by fragmentation-weighted operations. In: Jansson J, Martín-Vide C, Vega-Rodríguez M (eds) Algorithms for computational biology, vol 10849. Springer, Heidelberg, pp 53–64

Alexandrino AO, Miranda GHS, Lintzmayer CN, Dias Z (2019) Approximation algorithms for sorting permutations by length-weighted short rearrangements. Electron Notes Theor Comput Sci 346:29–40

Alexandrino AO, Lintzmayer CN, Dias Z (2020) Sorting permutations by fragmentation-weighted operations. J Bioinform Comput Biol 18(2):2050006.1–2050006.31

Bader M, Ohlebusch E (2007) Sorting by weighted reversals, transpositions, and inverted transpositions. J Comput Biol 14(5):615–636

Bender MA, Ge D, He S, Hu H, Pinter RY, Skiena SS, Swidan F (2008) Improved bounds on sorting by length-weighted reversals. J Comput Syst Sci 74(5):744–774

Berman P, Hannenhalli S, Karpinski M (2002) 1.375-Approximation algorithm for sorting by reversals. In Möhring R, Raman R (ed) Proceedings of the 10th annual European symposium on algorithms (ESA'2002), volume 2461 of lecture notes in computer science. Springer, Heidelberg, pp 200–210

Blanchette M, Kunisawa T, Sankoff D (1996) Parametric genome rearrangement. Gene 172(1):GC11–GC17

Bulteau L, Fertin G, Rusu I (2012) Sorting by transpositions is difficult. SIAM J Discrete Math 26(3):1148–1180

Caprara A (1999) Sorting permutations by reversals and Eulerian cycle decompositions. SIAM J Discrete Math 12(1):91–110

Chen X (2013) On sorting unsigned permutations by double-cut-and-joins. J Combin Optim 25(3):339–351

Dias UM (2012) Problemas de Comparação de Genomas. PhD thesis, Institute of Computing, University of Campinas. (**In Portuguese**)

Elias I, Hartman T (2006) A 1.375-approximation algorithm for sorting by transpositions. IEEE/ACM Trans Comput Biol Bioinform 3(4):369–379

Eriksen N (2002) $(1+\epsilon)$-approximation of sorting by reversals and transpositions. Theor Comput Sci 289(1):517–529

Galvão GR, Lee O, Dias Z (2015) Sorting signed permutations by short operations. Algorithms Mol Biol 10(1):1–17

Hannenhalli S, Pevzner PA (1999) Transforming cabbage into turnip: polynomial algorithm for sorting signed permutations by reversals. J ACM 46(1):1–27

Heath LS, Vergara JPC (2003) Sorting by short swaps. J Comput Biol 10(5):775–789

Jerrum MR (1985) The complexity of finding minimum-length generator sequences. Theor Comput Sci 36(2–3):265–289

Jiang H, Zhu D, Zhu B (2012) A $(1+\epsilon)$-approximation algorithm for sorting by short block-moves. Theor Comput Sci 437:1–8

Jiang H, Feng H, Zhu D (2014) An 5/4-approximation algorithm for sorting permutations by short block moves. In Ahn H, Shin C (ed) Proceedings of the 25th international symposium on algorithms and computation (ISAAC'2014), volume 8889 of lecture notes in computer science. Springer, Heidelberg, pp 491–503

Lefebvre J-F, El-Mabrouk N, Tillier ERM, Sankoff D (2003) Detection and validation of single gene inversions. Bioinformatics 19(1):i190–i196

Lintzmayer CN, Fertin G, Dias Z (2015) Approximation algorithms for sorting by length-weighted prefix and suffix operations. Theor Comput Sci 593:26–41

Lintzmayer CN, Fertin G, Dias Z (2018) Sorting permutations and binary strings by length-weighted rearrangements. Theor Comput Sci 715:35–59

Miranda GHS, Lintzmayer CN, Dias Z (2019) Sorting permutations by λ-operations. J Univ Comput Sci 25(2):98–121

Nguyen TC, Ngo HT, Nguyen NB (2005) Sorting by restricted-length-weighted reversals. Genomics Proteomics Bioinform 3(2):120–127

Oliveira AR, Brito KL, Dias U, Dias Z (2019) On the complexity of sorting by reversals and transpositions problems. J Comput Biol 26(11):1223–1229

Pinter RY, Skiena S (2002) Genomic sorting with length-weighted reversals. Genome Inform 13:103–111

Rahman A, Shatabda S, Hasan M (2008) An approximation algorithm for sorting by reversals and transpositions. J Discrete Algorithms 6(3):449–457

Swidan F, Bender MA, Ge D, He S, Hu H, Pinter RY (2004) Sorting by length-weighted reversals: dealing with signs and circularity. In: Sahinalp S, Muthukrishnan S, Dogrusoz U (eds) Combinatorial pattern matching, vol 3109. Lecture notes in computer science. Springer, Heidelberg, pp 32–46

Vergara JPC (1998) Sorting by bounded permutations. PhD thesis, Virginia Polytechnic Institute and State University

Walter MEMT, Dias Z, Meidanis J (1998) Reversal and transposition distance of linear chromosomes. In Proceedings of the 5th international symposium on string processing and information retrieval (SPIRE'1998), Los Alamitos, CA. IEEE Computer Society, pp 96–102

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Affiliations

Alexsandro Oliveira Alexandrino[1] · Guilherme Henrique Santos Miranda[1] · Carla Negri Lintzmayer[2] · Zanoni Dias[1]

✉ Alexsandro Oliveira Alexandrino
alexsandro@ic.unicamp.br

Guilherme Henrique Santos Miranda
guilherme.miranda@students.ic.unicamp.br

Carla Negri Lintzmayer
carla.negri@ufabc.edu.br

Zanoni Dias
zanoni@ic.unicamp.br

1 Institute of Computing, University of Campinas, Campinas, SP, Brazil

2 Center for Mathematics, Computation and Cognition, Federal University of ABC, Santo André, SP, Brazil